

Regression Models Using Gradient Descent

Linear through sixth-degree polynomial regression trained from scratch, with weight update derivations, results, and a direct comparison against the Normal Equation solution from Assignment 02.

STUDENT

Muhammad Zohaib Khan

ROLL NUMBER

BSDSF24M016

PROGRAMME

BS Data Science, 5th Semester

DATE

4 October 2026



Contents & Summary

01	Introduction	2
02	Dataset and Experimental Setup	2
03	Cost Function and Weight Update Equations	3
04	Results	4
05	Gradient Descent vs. Normal Equation	5
06	Convergence Analysis	5
07	Overfitting and Underfitting	6
08	Conclusion	6

At a glance

0.1086BEST GD TEST MSE
(DEGREE 6)**0.0223**BEST NORMAL EQ.
TEST MSE (DEGREE
6)**0.0863**GAP AT DEGREE 6
(GD - NE)**≈ 0.002**AGREEMENT FOR
DEGREES 1-2

Six polynomial models were trained with batch gradient descent (learning rate 0.01, 1000 iterations). Linear and quadratic fits agree with the closed-form solution to within 0.002. Degrees 3 to 6 are still converging at the final iteration, so their error is well above the Normal Equation optimum. No overfitting was observed, because every model is underfit at this iteration budget.

Convention. All MSE values in this report use the course convention $J = (1 / 2m) \sum (h - y)^2$, the same as the notebook.

01 Introduction

This assignment implements polynomial regression models using gradient descent (GD), an iterative optimisation method, in place of the closed-form Normal Equation used in Assignment 02. The goal is to fit models of degree 1 through 6 to a single-feature dataset (input X , target R), with the optimisation written from scratch in NumPy. Every model is then compared with its Normal Equation counterpart on both train and test data.

02 Dataset and Experimental Setup

Parameter	Value	Notes
Training samples (m)	283	trainRegression.csv
Test samples	32	testRegression.csv
Learning rate (α)	0.01	Fixed across all degrees
Iterations	1000	Batch gradient descent
Initial weights	$\theta = 0$	Zero vector of length $n+1$
Feature scaling	None (deg 1–2)	z-score for deg 3–6

For degrees 3 to 6, each power of x is standardised using the mean and standard deviation of the **training** set, and the same statistics are applied to the test set. This avoids leaking information from the test set into training. Normalisation matters because x^6 spans a much wider range than x , which would force a very small learning rate on some weights and a very slow one on others.

03

Cost Function and Weight Update Equations

3.1 General form

For hypothesis $h_{\theta}(x) = X\theta$, the cost function is:

COST FUNCTION

$$J(\theta) = (1 / 2m) \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Differentiating with respect to each weight gives the gradient, and the update applied at every iteration in vectorised form is:

GRADIENT DESCENT UPDATE

$$\theta \leftarrow \theta - \alpha \cdot (1 / m) \cdot X^T (X\theta - y)$$

3.2 Update equations for each model

Model 1 — Linear

2 weights

$$h(x) = \theta_0 + \theta_1 x$$

$$\theta_0 \leftarrow \theta_0 - \alpha \cdot (1/m) \sum (h - y)$$

$$\theta_1 \leftarrow \theta_1 - \alpha \cdot (1/m) \sum (h - y) x$$

Model 2 — Quadratic

3 weights

$$h(x) = \theta_0 + \theta_1 x + \theta_2 x^2$$

$$\theta_j \leftarrow \theta_j - \alpha \cdot (1/m) \sum (h - y) x^j \quad \text{for } j = 0, 1, 2 \text{ (with } x^0 = 1)$$

Model 3 — Cubic

4 weights, normalised

$$h = \theta_0 + \theta_1 z_1 + \theta_2 z_2 + \theta_3 z_3, \quad z_k = (x^k - \mu_k) / \sigma_k$$

$$\theta_j \leftarrow \theta_j - \alpha \cdot (1/m) \sum (h - y) z_j \quad \text{for } j = 0 \text{ to } 3 \text{ (} z_0 = 1)$$

Models 4, 5 and 6 — General degree n

n+1 weights

$$h = \sum_{j=0}^n \theta_j z_j \Rightarrow \theta_j \leftarrow \theta_j - \alpha \cdot (1/m) \sum_i (h^{(i)} - y^{(i)}) z_j^{(i)}$$

Degree 4 updates 5 weights, degree 5 updates 6, and degree 6 updates 7, all simultaneously in each iteration.
Assignment 03 · Gradient Descent

04 Results

4.1 Learned weights

Model	θ_0	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6
1 · Linear	0.7512	-0.4903	–	–	–	–	–
2 · Quadratic	0.5817	-0.1750	-0.0974	–	–	–	–
3 · Cubic	0.0049	-0.5215	-0.2257	0.2839	–	–	–
4	0.0049	-0.5496	-0.3794	0.0417	0.4505	–	–
5	0.0049	-0.5134	-0.4603	-0.1325	0.2056	0.4892	–
6	0.0049	-0.4538	-0.4844	-0.2321	0.0440	0.2777	0.4603

Models 1–2 use raw features, so their weights are on the original scale. Models 3–6 use normalised features, so θ_0 equals the mean training target (≈ 0.0049) and the other weights are not directly comparable with Models 1–2.

4.2 Error and convergence

Model	Train MSE	Test MSE	Cost @ 100	Cost @ 500	Cost @ 1000
1 · Linear	0.1514	0.1571	0.2211	0.1638	0.1514
2 · Quadratic	0.1619	0.1610	0.1958	0.1696	0.1619
3 · Cubic	0.1365	0.1485	0.1654	0.1475	0.1365
4	0.1151	0.1332	0.1674	0.1297	0.1151
5	0.0980	0.1192	0.1638	0.1110	0.0980
6	0.0871	0.1086	0.1566	0.0968	0.0871

MSE uses the $(1/2m)$ convention. Cost columns are the values of $J(\theta)$ at the given iteration.

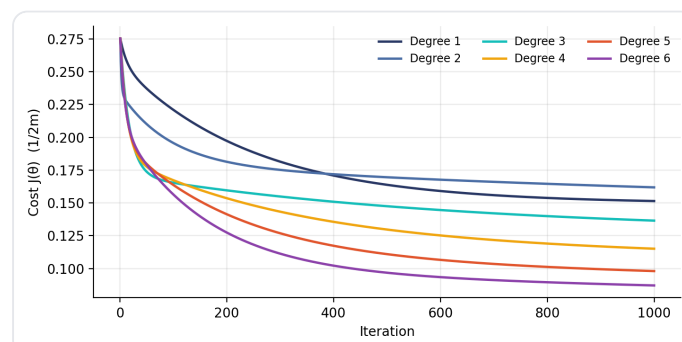


Figure 1. Cost $J(\theta)$ against iteration for all six models. Linear and quadratic flatten early; degrees 3–6 are still descending at iteration 1000.

05 Gradient Descent vs. Normal Equation

Assignment 02 solved the same problems in closed form, $\theta = (X^T X)^{-1} X^T y$. The table below places both approaches on the same (1/2m) scale. Assignment 02 values were originally reported as (1/m) and have been halved here.

Model	GD Train	NE Train	GD Test	NE Test	Test gap (GD – NE)	Ratio
1 · Linear	0.1514	0.1496	0.1571	0.1580	−0.0009	0.99×
2 · Quadratic	0.1619	0.1442	0.1610	0.1630	−0.0020	0.99×
3 · Cubic	0.1365	0.0249	0.1485	0.0258	+0.1227	5.8×
4	0.1151	0.0222	0.1332	0.0250	+0.1082	5.3×
5	0.0980	0.0193	0.1192	0.0221	+0.0971	5.4×
6	0.0871	0.0193	0.1086	0.0223	+0.0863	4.9×

Gap = GD test MSE minus Normal Equation test MSE. Ratio = GD test MSE divided by Normal Equation test MSE.

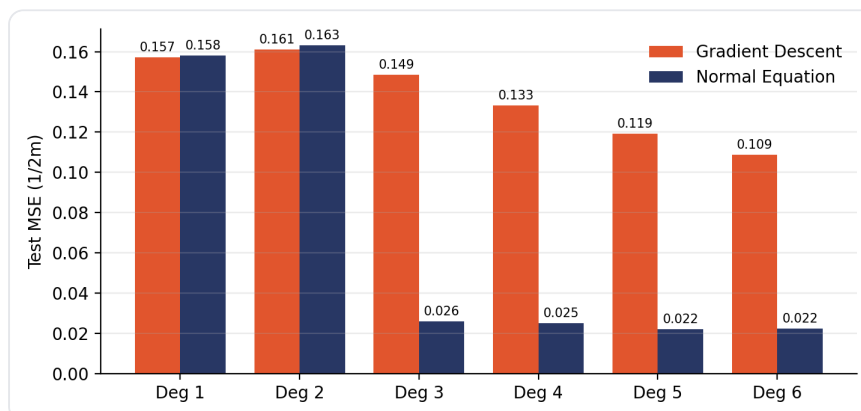


Figure 2. Test MSE by degree, gradient descent against the Normal Equation.

Reading the gap. For linear and quadratic regression, GD reaches the least-squares minimum almost exactly. For degrees 3–6, GD is five to six times worse. The Normal Equation finds the exact minimum in one step, whereas GD only approaches it. The cost curves are still descending at iteration 1000, so the gap reflects the learning rate and iteration budget rather than a weakness of the models.

06 Convergence Analysis

Fast

Models 1 and 2 flatten within the first 500 iterations.

Still falling

Models 3–6 are descending at iteration 1000.

Scaling is key

Normalisation lets one learning rate serve all weights.

Next step

Raise α or run more iterations to reach the minimum.

- **Models 1 and 2** lose about 0.06 of cost in the first 500 iterations, then change very little. They are close to their minimum by iteration 1000.
- **Models 3 to 6** are still falling steadily. Model 6 drops from 0.0968 to 0.0871 between iterations 500 and 1000, so more training would lower the error further.
- Without normalisation, the large values of x^5 and x^6 would make a single learning rate unusable for every weight at once.

07 Overfitting and Underfitting

The gradient descent models are **underfit**, not overfit. Train and test MSE both decrease as the degree rises, and test error never sits far above train error. Because each model stopped before convergence, its extra flexibility is only partly used.

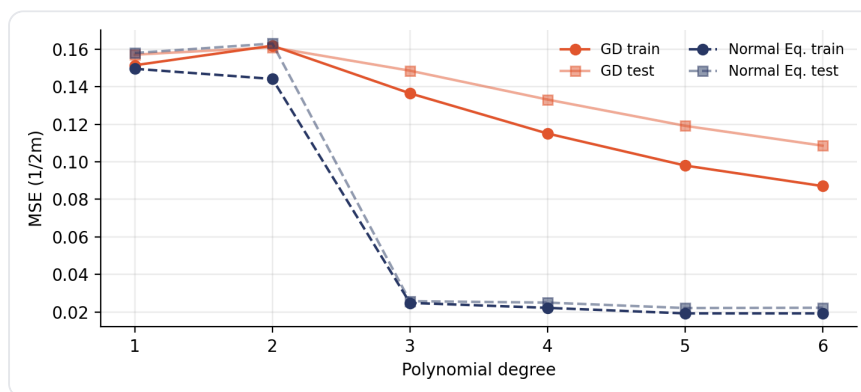


Figure 3. Train and test MSE against polynomial degree. Solid and dashed lines mark GD and Normal Equation respectively. The overfitting rise seen in Assignment 02 at degree 6 cannot appear here, because the models have not yet fitted the training noise.

08 Conclusion

The implementation reproduces the Normal Equation solution for linear and quadratic regression within 0.002, which confirms that the update equations in Section 3 are correct. For cubic through sixth-degree models, gradient descent with $\alpha = 0.01$ and 1000 iterations has not converged, so its errors are substantially higher than the closed-form values.

The Normal Equation is exact and efficient for a small number of features. Gradient descent is the better choice when the feature count or dataset is large enough that inverting $X^T X$ becomes expensive, but it needs a sensible learning rate, feature scaling, and enough iterations to match the exact solution.